Язык программирования D

Основы синтаксиса и запуск

Выполнили студенты группы 5030102/30401
Мугтасимова Лиана и Коваль Егор

Что такое D?

Компилируемый язык общего назначения с сильной системной составляющей

Похожий синтаксис

Фигурные скобки, функции, циклы и привычные типы: вход в язык понятен после C/C++.

Современные механизмы

Вывод типов, динамические массивы, модули, сборщик мусора и работа с низкоуровневыми данными.

Как запустить программу?

Установить DMD или открыть онлайн-среду

D

```d
import std.stdio;

void main()
{
    writeln("Hello, D!");
}
```

Локально

dmd hello.d - исполняемый файл
rdmd hello.d - сборка и запуск

В браузере

run.dlang.io - без установки

Результат

Hello, D!

Анатомия D-программы

Из чего состоит минимальное приложение

D

```
import std.stdio;  // ввод и вывод

void main()       // точка входа
{
    writeln("Hello, D!");
}
```

import

Подключает модуль стандартной библиотеки

main

Функция, с которой начинается выполнение

writeln

Выводит данные и переводит строку

Типы, auto и immutable

Типизируем данные явно — или позволяем компилятору вывести тип

D

```
int age = 19;
double average = 4.75;
auto name = "Anna";
immutable int passingScore = 3;

writeln(name, ": ", average);
```

int / double

Целое и вещественное число

auto

Тип определяется по выражению справа

immutable

Значение нельзя изменить после инициализации

Операции и приведение типов

Почему 7 / 3 и cast(double) 7 / 3 дают разные результаты

D

```
int sum = 7 + 3;
int division = 7 / 3;
double exact = cast(double) 7 / 3;
bool ok = sum > 8 && division == 2;

writeln(division); // 2
writeln(exact);    // 2.33333...
```

Целочисленное деление

7 / 3 → 2

Явное преобразование

cast(double) 7 → 7.0

Логические операции

&&, ||, !

Выбор пути: if / else

Управление поведением программы на основе балла

D

```
int score = 78;

if (score >= 85)
    writeln("Excellent");
else if (score >= 60)
    writeln("Good");
else
    writeln("Not good");
```

Что произойдёт?

78 < 85, но 78 >= 60

Вывод

Good

Ещё одна конструкция

switch удобен для выбора из фиксированных вариантов

for и foreach

Обходим массив без ручного управления индексом

D

```
int[] scores = [5, 4, 3, 5];
int total = 0;

foreach (score; scores)
    total += score;

for (int i = 0; i < 3; ++i)
    writeln("Step ", i);
```

foreach

Берёт очередной элемент коллекции

for

Полезен, если нужен индекс / счётчик

Результат

total = 17

Динамические массивы

Размер можно увеличивать прямо во время выполнения

D

```
int[] scores = [5, 4, 3];
scores ~= 5;

writeln(scores);
writeln(scores.length);
writeln(scores[0]);
```

int[]

Динамический массив целых

~=

Присоединяет элемент (или массив)

Вывод

```
[5, 4, 3, 5]
4
5
```

Передаём массив — получаем результат

У функций есть аргументы и возвращаемое значение

D

```
double average(const int[] scores)
{
    int total = 0;
    foreach (score; scores)
        total += score;
    return cast(double) total
          / scores.length;
}
```

const int[]

Функция не изменяет элементы массива

double

Тип возвращаемого значения

Пример

average([5, 4, 3, 5]) → 4.25

Функция вызывает саму себя

Пример: факториал с условием остановки

D

```
ulong factorial(uint n)
{
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

writeln(factorial(5)); // 120
```

Базовый случай

n <= 1 → 1

Рекурсивный шаг

n * factorial(n − 1)

Раскрытие

5 × 4 × 3 × 2 × 1 = 120

Где доступна переменная?

Блок в фигурных скобках задаёт локальную область

D

```
int value = 10;
{
   int local = 5;
   writeln(value + local); // 15
}

writeln(value);          // 10
// writeln(local);       // ошибка
```

Внутри блока

Виден value и local

Снаружи блока

local уже недоступна

Практический смысл

Меньше случайных конфликтов имён

Анализ оценок студента

Объединяем массивы, функции, условия и циклы

D

```
int[] grades = [5, 4, 3, 5, 4];
double avg = average(grades);

writeln("Grades: ", grades);
    writeln("Average grade: ", avg);
    writeln("Maximum: ", maximum(grades));
    writeln(avg >= 4.5 ? "High result" : "could be
better");
```

Что ещё есть в файле?

average() и maximum()

Проверка

assert защищает от пустого массива

Где исходник?

src/grades.d

Что вывела программа?

Код работает с исходными данными и печатает понятный отчёт

```
> rdmd grades.d

Grades: [5, 4, 3, 5, 4]
Average grade: 4.2
Maximum: 5
could be better
```

ИТОГ: D позволяет писать привычный код, но добавляет удобные средства для работы с типами и коллекциями.